

Resumen ejecutivo

FASTBOOK 05 — Core Desarrollo

CI/CD: herramientas para la integración
y el despliegue continuos

Autores del documento original: Juan Arias, Arturo Mora y Jaime Morales

Este resumen sintetiza los conceptos clave del fastbook original: la cultura DevOps, los conceptos de integración y despliegue continuos (CI/CD), la estructura de los pipelines, Jenkins como herramienta central, y otras herramientas complementarias (Maven y XRay).

1. La cultura DevOps

DevOps (development + operations) es una revolución cultural y técnica en la forma de trabajar de equipos y organizaciones, que promueve la colaboración entre gestión, desarrollo y operaciones.

¿Por qué las empresas adoptan DevOps?

Los sistemas de trabajo tradicionales generan largos plazos de entrega que:

- Impiden ofrecer servicios de vanguardia.
- Impiden a los equipos de TI trabajar de forma ágil.
- Desgastan la experiencia del cliente.

Tres factores clave de la revolución cultural DevOps:

- **Colaboración e interacción abierta** entre equipos de operaciones y desarrollo.
- **Derribar los silos**, barreras artificiales que frenan la colaboración.
- **Ciclos cortos** de diseño, implementación y despliegue.

El ciclo de vida DevOps se representa habitualmente como un bucle infinito: *plan* → *create* → *verify* → *package (DEV)* y *release* → *configure* → *monitor (OPS)*, con el objetivo de producir software de alta calidad dentro de plazos y costes controlados.

Las empresas que adoptan DevOps suelen combinarlo con metodología **Agile**, que permite adaptar rápidamente estimaciones, requisitos y alcance del proyecto.

2. CI/CD: integración y despliegue continuos

CI/CD se apoya en un método para distribuir aplicaciones a los clientes de forma automatizada, pasando por todas las etapas de desarrollo, con el objetivo de lograr integración, distribución y despliegue continuos.

Antipatrones que CI/CD busca evitar

- **Despliegue manual del software:** pasos atómicos ejecutados por personas distintas, lo que genera despliegues no deterministas.
- **Despliegue de producción solo al final del desarrollo:** operaciones no participa hasta el final, lo que genera scripts sin probar y baja fiabilidad.
- **Gestión manual de configuración en producción:** introduce riesgo, ya que los cambios pueden perderse o no quedar documentados.

La solución común a estos antipatrones es la automatización completa: la intervención humana debería limitarse a seleccionar entorno, seleccionar versión y confirmar el despliegue.

Definiciones clave

Integración continua (Martin Fowler): “Los miembros de un equipo integran su trabajo de manera frecuente, incluso varias veces al día. Cada integración se verifica con una construcción y pruebas automáticas para detectar errores tan pronto como sea posible.”

Entrega continua (Jez Humble): “La capacidad de llevar los cambios de un sistema hasta el entorno de producción de forma segura y rápida, para que los despliegues se puedan llevar a cabo de manera predecible y rutinaria siempre que haga falta.”

Despliegue continuo: capacidad de desplegar en producción múltiples veces al día. No siempre coincide con la entrega continua: puede haber motivos de negocio para no desplegar tras cada integración (fechas de lanzamiento, paradas de servicio, versiones de software discreto).

Madurez de la organización

Basado en el modelo de Jez Humble y David Farley (*Continuous Delivery*, 2010), permite identificar el nivel de madurez de una organización y guiar su progreso. Objetivos deseables:

- Reducir el tiempo entre despliegues (cycle time).
- Reducir incidentes y tiempo de soporte.
- Hacer el ciclo de vida del software más predecible.
- Adaptarse a regulaciones de forma natural.
- Gestionar riesgos de forma efectiva.
- Reducir costes gracias a una buena gestión del riesgo.

La escala de madurez va de **-1 (Regresión)** a **3 (Optimización)**, evaluando cinco áreas: construcción y CI, entornos y despliegues, liberación de versiones, pruebas y gestión de bases de datos.

3. Pipelines de CI/CD

Un **pipeline** encadena las fases de compilación, empaquetado, pruebas y despliegue, cumpliendo dos características fundamentales: el movimiento automático del código/artefactos entre fases, y la repetición continua del flujo (por eventos o de forma temporizada).

Fases comunes de un pipeline

- **1. Validación y construcción:** iniciada por cambios en el repositorio (commits, pull requests, tareas planificadas). Incluye análisis estático, tests unitarios, descarga de dependencias y compilación. Principio clave: fallar lo antes posible y dar el máximo feedback.
- **2. Publicación de artefactos:** almacenamiento de las versiones generadas en almacenes especializados (Docker Registry, Maven Central, PyPI) o unificados como Artifactory o Nexus. Se valoran seguridad y conectividad.
- **3. Control de calidad:** análisis con herramientas como SonarQube (deuda técnica, código duplicado), análisis de vulnerabilidades (p. ej. Docker Scan, referencias OWASP) y test de integración.
- **4. Despliegue:** se distingue entre despliegues *sin downtime* (sin pérdida de servicio) y *con downtime* (requieren reinicio, mayor riesgo de pérdida de datos).

Es habitual inyectar *flags* según el tipo de build (pull request, rama de desarrollo, nightly, master/main) para decidir si se genera artefacto, se ejecutan tests costosos o se promociona a producción.

4. Jenkins, herramienta clave para CI/CD

Jenkins es una herramienta de automatización de código abierto, evolucionada a partir de Hudson (Sun Microsystems). Su idea central es la creación de **trabajos (jobs)** que ejecutan operaciones como compilación, pruebas o despliegue, y que pueden encadenarse entre sí.

Arquitectura distribuida

- **Servidor principal (master):** controla el sistema, aloja configuración, definición de trabajos e historial. En entornos profesionales se recomienda que solo distribuya tareas.
- **Agentes (antes “esclavos”):** ejecutan las tareas delegadas por el master. Pueden ser estáticos o dinámicos (desplegados bajo demanda en la nube), y de distintos sistemas operativos.
- Cada agente dispone de uno o varios **ejecutores**, que permiten correr varios trabajos en paralelo.

Estructura de un trabajo

- Uno o varios *triggers* (disparadores): periódicos, manuales, desde otro trabajo o por *webhook*.
- Lista de agentes permitidos (mediante etiquetas).
- Parámetros de entrada.
- Tareas a ejecutar (pueden arrancar otros trabajos).
- Tareas de finalización (postprocessing).
- Artefactos archivados.

Los **webhooks** permiten arrancar trabajos de forma asíncrona (p. ej. desde GitHub al abrir una pull request), evitando el modelo síncrono de *polling*, que sobrecarga tanto Jenkins como el sistema de control de versiones.

Jenkinsfiles

Archivo escrito en Groovy que define el trabajo (fases, scripts, etc.) y puede versionarse junto al código de la aplicación. Admite dos sintaxis:

- **Sintaxis en script:** imperativa, más versátil pero requiere más código; el usuario controla flujo y errores.
- **Sintaxis declarativa:** introducida en Jenkins 2, basada en un DSL sobre Groovy; más fácil de leer y delega el control de flujo y errores en el propio motor de Jenkins.

Un Jenkinsfile declarativo típico define: agente, triggers, variables de entorno y *stages* (por ejemplo: Build, Push, Unit tests), cada uno con sus propios *steps*. Se pueden añadir condicionales (*when*) para ejecutar pasos solo en ciertas ramas.

Gestión de secretos

Jenkins permite almacenar credenciales sensibles a nivel global o restringidas a proyectos concretos: texto en claro, pares usuario/contraseña, ficheros (p. ej. kubeconfig), claves SSH y certificados. Esto evita exponer secretos directamente en los repositorios.

Jenkins y contenedores

Como alternativa a agentes físicos preconfigurados, Jenkins puede usar imágenes Docker para proveer entornos de trabajo, incluyendo la posibilidad de construir el pipeline directamente a partir de un Dockerfile presente en el repositorio.

5. Otras herramientas: Maven y XRay

Apache Maven

Proyecto que estandariza la configuración de un proyecto Java a lo largo de todo su ciclo de vida: compilación, empaquetado, gestión de dependencias y distribución de librerías. Creado en 2002, su configuración se define en el archivo **pom.xml**.

Fases principales del ciclo de vida:

- **Compile:** genera los .class a partir de los .java.
- **Test:** ejecuta los tests automáticos JUnit.
- **Package:** genera el .jar con los .class compilados.
- **Install:** copia el .jar al repositorio local.
- **Deploy:** copia el .jar a un servidor remoto.

Funcionalidades adicionales: *Clean* (limpia artefactos generados), *Assembly* (genera un .zip de instalación), *Site* y *Site-deploy* (generación y publicación de un sitio web del proyecto).

El archivo **pom.xml** incluye elementos como *project*, *modelVersion*, *groupId*, *artifactId* y *version*, y permite definir dependencias, plugins y perfiles de compilación.

XRay

Herramienta integrada con Jira orientada a la planificación, diseño, ejecución de pruebas y generación de informes. Utiliza tipos de incidencia específicos en cada fase:

- **Planificación:** incidencias de planificación de pruebas.
- **Diseño:** incidencias de condición previa y de prueba; organización mediante conjuntos de pruebas.
- **Ejecución:** incidencias de ejecución de la prueba.
- **Generación de informes:** informes de cumplimiento de requisitos e incidencias personalizadas mediante Jira Software.

XRay resulta especialmente útil para añadir pruebas *ad hoc* y no planificadas incluso en proyectos pequeños.

6. Conclusión

- El fastbook aborda el proceso de integración y despliegue continuos que actualmente implementan la mayoría de las compañías, definiendo su terminología clave.
- Jenkins es presentada como la herramienta “padre” de CI/CD, complementada por Maven (gestión de dependencias) y XRay (gestión de pruebas), que pueden integrarse con Jenkins.
- Jenkins no se limita a automatizar pruebas de funcionalidad: suele integrarse con otras herramientas, como Kiuwan para análisis de seguridad del código.
- Como lectura complementaria se recomienda la documentación oficial de Jenkins, especialmente la sección dedicada a Pipeline.

Idea central: automatizar por completo el ciclo de construcción, prueba y despliegue reduce el error humano, aumenta la fiabilidad y permite entregar valor al cliente de forma más rápida, frecuente y predecible.